Mikrocomputertechnik 1

Einheit 5: Der Single-Cycle-Datenpfad

Prof. Dr.-Ing. Daniel Klünder

Folie 1 / 79

Der Single-Cycle-Datenpfad

Notizen

Diese Einheit gehört zu Mikrocomputertechnik 1.
Sie erklärt, wie die schon bekannten Befehle in einem einfachen Single-Cycle-Modell in einer Taktperiode umgesetzt werden.
Single Cycle heißt hier: zwischen zwei aktiven Flanken wird genau eine Instruktion kombinatorisch vorbereitet und an der nächsten Flanke als neuer Zustand übernommen.
Beispiel: add t0, t1, t2 ist der rote Faden, noch ohne Schaltbild.
Als Nächstes geht es von der Programmiersicht zu den Bausteinen, die Register und Speicher ändern. Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 2 / 79

Von der Instruktion zur Hardware

- Herzlich willkommen zur fünften Vorlesung in Mikrocomputertechnik 1
- Bisher: Fokus auf die Software-Sicht (Assembler)
- Heute: Brückenschlag zurück zur Hardware
- Die Blackbox des RISC-V-Prozessors öffnen
- Ziel: aus dem Maschinenwort werden Logikpegel, die Quellen und Schreibfreigaben wählen



Notizen

Assemblertext, Maschinenwort und gespeicherter Zustand sind drei Ebenen.
Der Assembler erzeugt ein 32-Bit-Wort. Die Hardware wertet dessen Bits aus und setzt damit Logikpegel für Operation, Quellen und Schreibfreigaben. Das ist kein Stromschalter je Textzeile.
CPU heißt Central Processing Unit. RISC-V ist der Name der offenen Befehlssatzarchitektur.
Beispiel: Bei add t0, t1, t2 liegen zwei Registerwerte an der ALU (Arithmetic Logic Unit); das Ergebnis wird erst mit der Flanke nach t0 übernommen, wenn die Freigabe stimmt.
Die Agenda ordnet holen, lesen, rechnen, speichern und zurückschreiben.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 3 / 79

Agenda für den heutigen Tag

1. Die 5 Phasen — Lebenszyklus eines Maschinenbefehls
2. Fetch und R-Type — Hardware für Berechnungen
3. Load und Store — Arbeitsspeicher physikalisch einbinden
4. Der Branch-Pfad — dem PC das Springen beibringen
5. Das Gesamtbild — Zusammenführung über Multiplexer
6. Ripes — visuelles Tracing auf den Logikgattern



Notizen

Jeder Agendapunkt beantwortet eine Frage: Woher kommt der Befehl, woher die Werte, wie wird gerechnet, wie werden Speicher und nächste Adresse gewählt?
R-Type ist das Register-Register-Format. Ein Multiplexer wählt eine Datenquelle. Ripes zeigt eine simulierte Mikroarchitektur, kein analoges Messen an echten Gattern.
Lernziele: einen add-, lw- und sw-Pfad erklären, die nächste PC-Adresse bei beq bestimmen und Drahtwert vor der Flanke vom Zustand danach trennen.
Beispiel: Die kleine Grafik ist nur die Reihenfolge der Themen.
Zuerst werden die bekannten Befehlsformate für den Hardwarepfad aktiviert.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 4 / 79

Rückblick: Was wir bisher blind genutzt haben

- Maschinenbefehl immer exakt 32 Bit breit
- Instruction Formats: R, I, S, B
- Register lesen und überschreiben (`t0`, `s0`, …)
- ALU kann addieren und vergleichen
- PC als logischer Zeiger im Code

```
# Wir wissen, WAS dieser Befehl tut:
add t0, t1, t2

# Aber WIE wird aus 0x007302B3 die ALU aktiviert?
```

Notizen

ALU, CPU und PC (Program Counter, Adresse der aktuellen Instruktion) sind hier die Arbeitsbegriffe.
Immer 32 Bit gilt für dieses unkomprimierte RV32I-Modell, nicht für RISC-V mit komprimierten Instruktionen.
R, I, S und B sind die hier nötigen Formate, nicht alle Formate der ISA.
0x007302B3 ist add t0, t1, t2: rd=x5, rs1=x6, rs2=x7, opcode=0x33, funct3=0, funct7=0. Die Hexzahl 0x kennzeichnet die Basis 16. Die Felder sind Nummern, nicht die Datenwerte.
Beispiel: Der Assembler hat den Text schon verlassen; die Hardware sieht nur das Wort.
Wir öffnen die Blackbox als Blockdiagramm, nicht als Milliarden Transistoren.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 5 / 79

Die Blackbox öffnen

- Ein realer Prozessor: Milliarden Transistoren
- Mikroskopische „Autobahnen“ — für Menschen nicht greifbar
- Abstraktion: Blockdiagramme statt einzelner Elektronen
- Fokus: Datenflüsse und Vernetzung der Komponenten

Notizen

Ein Blockdiagramm abstrahiert Funktion und Ports. Reale Kerne haben sehr unterschiedliche Transistorzahlen; eine feste Milliardenzahl gilt nicht für jeden Mikrocontroller.
Registerbank, ALU, Speicher und MUX sind funktionale Bausteine. Ein 32-Bit-Datenwort, eine 5-Bit-Registernummer und ein 1-Bit-Steuersignal sind verschiedene Leitungsbreiten.
Die Ports genügen, um den Datenfluss zu begründen, ohne jedes Gatter zu zeichnen.

Beispiel: A1=5 adressiert ein Register; der gelesene Wert kann 10 sein. Beides ist nicht dieselbe Leitung. Diese verbundenen Verarbeitungsbausteine heißen Datenpfad.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 6 / 79

Was ist ein Datenpfad (Datapath)?

- Physikalische „Rohrleitung“ des Prozessors
- Alles, was Daten verarbeitet oder transportiert: ALU, Register File, Speicher, Multiplexer
- Nicht dazu: die Control Unit (Steuerwerk) — sie dirigiert nur
- Der Datenpfad begrenzt die Maximalleistung des Chips



Notizen

Der Datenpfad verarbeitet Werte. Das Steuerwerk wählt Operation, Quelle und Schreibfreigabe.
Eine MUX-Auswahl ist ein Steuersignal, das ausgewählte 32-Bit-Wort ist ein Datenwert. Beides gehört zum Prozessor; die folgenden Folien behandeln deshalb auch die nötigen Freigaben.
Verzögerungen in Daten- und Steuerpfad begrenzen die Taktperiode, zusammen mit den Zustandsspeichern.
Beispiel: ALUSrc=1 wählt ein Immediate, der Datenwert auf der Leitung kann trotzdem 12 sein.
Zuerst wird festgelegt, welche Befehle das Modell überhaupt beherrscht.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 7 / 79

Befehlsumfang dieses Lehrmodells

- Implementiert: `add`, `sub`, `and`, `or`, `lw`, `sw`, `beq`
- Kein vollständiger RV32I-Prozessor
- `addi`, `bne` und `jal` erst nach Ergänzung von ImmGen, Vergleich oder Link-Write-Back
- ImmGen liefert für `beq` den Byteoffset einschließlich implizitem Bit 0

Notizen

Das Basissubset ist add, sub, and, or, lw, sw und beq. RV32I ohne C meint hier 32-Bit-Instruktionen und einen sequenziellen Schritt von vier Byte.
addi kann den schon für lw vorhandenen Immediate- und Additionspfad nutzen, braucht aber passende Steuerung. bne braucht eine andere Branch-Entscheidung. jal braucht J-Immediate, unbedingte PC-Wahl und den Linkwert PC_alt+4. Alle drei bleiben außerhalb des Basissubsets, bis sie ergänzt sind.
Ripes kann mehr Befehle. Die spätere Tabelle ordnet Funktionen zu, sie behauptet keine baugleiche Zeichnung.
Beispiel: li und la in den Laboren sind Setup, keine nativen Basisbefehle.
Dieses Subset wird nun in genau einer Taktperiode je Instruktion umgesetzt.

Folie 8 / 79

Der Single-Cycle-Prozessor

- Viele Steuerungsarten: Pipelining, Multi-Cycle, …
- Heute: didaktisch zentrales Modell — Single-Cycle
- Ein Maschinenbefehl in exakt einem Taktzyklus
- Bei jeder steigenden Taktflanke ist ein Befehl fertig
- Leicht verständlich, modern oft ineffizient (Vertiefung später)

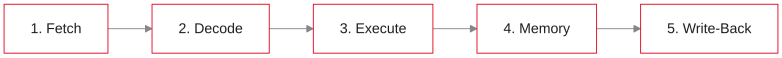
Notizen

Single Cycle, Multi Cycle und Pipelining sind drei Mikroarchitekturen. Hier liegt zwischen zwei Flanken genau eine Instruktion: Ausgangszustand, kombinatorische Rechnung, Übernahme des nächsten Zustands.
Ein Takt je Instruktion ist eine Modellannahme, keine ISA-Garantie. Kurze und lange Befehle teilen sich dieselbe Periodendauer.
Die fünf folgenden Namen sind Tätigkeiten in dieser einen Periode, nicht fünf Takte.
Beispiel: Nach der Flanke ist der PC neu, und die Leitungen gehören schon zur nächsten Instruktion.
Die Phasen beschreiben Inhalte, nicht fünf getrennte Clocks.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 9 / 79

Die 5 Phasen

- Logisch fünf Stationen, auch wenn alles in einem Takt geschieht
- Universelles Schema vieler Architekturen
- Datenfluss strikt von links nach rechts (Write-Back als Rückführung)
- Abkürzungen: IF, ID, EX, MEM, WB



Notizen

IF heißt Instruction Fetch, ID Instruction Decode, EX Execute, MEM Memory Access und WB Write-Back. Das ist eine didaktische Zerlegung, kein universeller Aufbau jeder CPU. Die Zeichnung von links nach rechts ist Orientierung; es gibt Rückführung und parallele Zweige.
add nutzt IF, ID, EX und WB, aber keinen Datenspeicher. lw nutzt zusätzlich MEM. sw schreibt Speicher und kein rd. beq ändert vor allem den PC.
Beispiel: Die fünf Kästen haben keine eigenen Taktflanken.
IF beginnt mit der Adresse der aktuellen Instruktion.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 10 / 79

Phase 1: Instruction Fetch (IF)

- 32-Bit-Maschinencode aus dem Speicher holen
- PC liefert im Lehrmodell die Instruktionsadresse; Adressübersetzung ist hier nicht modelliert
- Instruction Memory liefert die Instruktion
- Parallel: Vorbereitung $PC + 4$ für den nächsten Befehl



Notizen

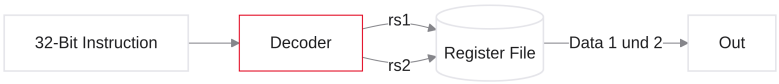
IF holt das Instruktionswort. Der PC liefert im Modell die Adresse; Adressübersetzung ist nicht enthalten. Adressweg und Instruktionswort sind beide 32 Bit breit, aber verschiedene Rollen.
Bei PC=0x100 wird das Wort an 0x100 gelesen. PC+4=0x104 ist ein Kandidat, noch nicht der gespeicherte neue PC. Ein Branch kann einen anderen Kandidaten wählen.
Beispiel: Das Lesen ändert das Instruktionswort nicht.
Das Maschinenwort wird als Nächstes in Felder und Steuerinformation zerlegt.

Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 11 / 79

Phase 2: Instruction Decode (ID)

- 32-Bit-Wort entschlüsseln, Operanden bereitstellen
- Felder: opcode, `rs1`, `rs2`, …
- `rs1`/`rs2` an das Register File — Werte auf die Datenleitungen
- Control Unit erzeugt Steuersignale aus dem Opcode



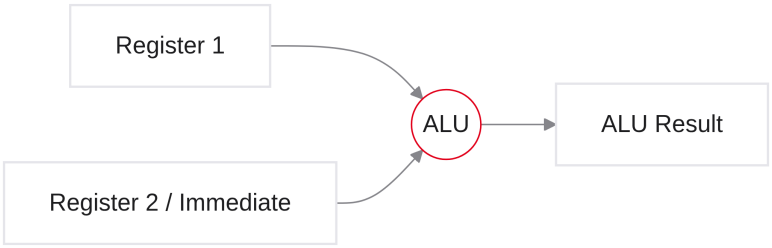
Notizen

ID wertet das Wort aus. opcode ist die Befehlsklasse. rs1 und rs2 sind 5-Bit-Nummern, RD1 und RD2 die zugehörigen 32-Bit-Werte.
add und sub haben denselben opcode und denselben funct3, aber verschiedenes funct7. Opcode allein unterscheidet sie nicht.
Decodierung und Registerlesen sind im Modell kombinatorisch, die Registerbank selbst speichert Zustand. Bereit heißt nach einer Verzögerung, nicht zeitlos.
Beispiel: A1=6 und A2=7 liefern die Inhalte von x6 und x7, nicht die Zahlen 6 und 7.
Mit den Operanden kann EX rechnen oder eine Adresse bilden.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 12 / 79

Phase 3: Execute (EX)

- Mathematische oder logische Arbeit: die ALU
- R-Type: zwei Registerwerte verknüpfen
- Load/Store: Zieladresse = Base + Offset
- Branch: Vergleich (z. B. Differenz auf Null)



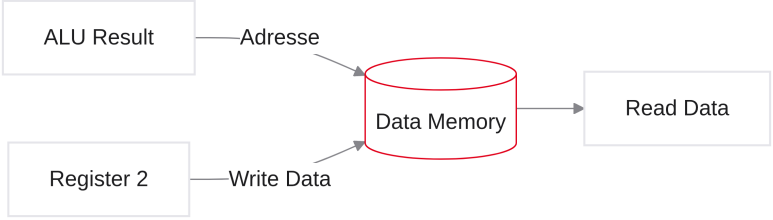
Notizen

EX ist die Rechenstufe. Dieselbe ALU hat im Modell drei Rollen: Datenoperation, effektive Adresse oder Differenz für beq.
10+15 ergibt 25, Basis plus 12 eine Adresse, 7−7 die Differenz 0. Die 32-Bit-Ausgabe ist je nach Befehl Ergebnis, Adresse oder ungenutzte Differenz. Zero ist ein zusätzliches Bit.
Die ALU führt nicht bei jedem Befehl dieselbe Operation aus. Die Zieladdition des Branches läuft im Lehrmodell in einem eigenen Addierer.
Beispiel: and und or sind bitweise Datenoperationen, nicht das einbitige Steuer-UND.
Eine Adresse ermöglicht danach den Datenspeicherzugriff.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 13 / 79

Phase 4: Memory (MEM)

- Kommunikation mit dem Datenspeicher
- Relevant für `lw`, `sw` und Verwandte
- Andere Befehle: Bypass am RAM vorbei
- `lw`: lesen; `sw`: schreiben an ALU-Adresse



Notizen

MEM ist der Datenspeicherzugriff, getrennt vom Instruktionslesen in IF.
lw liest das Wort an der ALU-Adresse. sw schreibt den Wert aus rs2 beziehungsweise RD2. add braucht keinen Datenspeicher; sein Ergebnis geht am Speicher vorbei. Nicht benutzt heißt nicht freigegeben, nicht dass jede Leitung null wäre.
Im Modell wird kombinatorisch gelesen und bei MemWrite an der Flanke geschrieben. Ein synchroner RAM mit zusätzlicher Leselatenz wäre ein anderes Modell.
Beispiel: Address, Write Data und Read Data sind drei Ports.
Ein Rechen- oder Load-Ergebnis muss bei Bedarf nach rd zurück.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 14 / 79

Phase 5: Write-Back (WB)

- Fertigen Wert in Zielregister `rd` ablegen
- Rückführung zum Register File (visuell „nach links“)
- `add`: ALU-Ergebnis; `lw`: RAM-Daten
- `beq`/`sw`: kein Register-Schreiben — Phase inaktiv



Notizen

WB schreibt einen Wert in das Zielregister rd, wenn RegWrite aktiv ist.
add schreibt das ALU-Ergebnis, lw das Speicherwort. sw und beq schreiben kein rd. Der Rückpfeil ist ein gerichteter Datenweg, keine Zeit rückwärts.
Vor der Flanke liegt der Wert am Schreibport. Erst die Flanke macht Zustand daraus. x0 bleibt null, auch wenn ein Schreibversuch ihn adressiert.
Beispiel: Bei add mit Ergebnis 25 und rd=t0 ist 25 vor der Flanke nur WD, danach der neue Inhalt.
Daraus wird jetzt ein konkreter Fetch- und R-Type-Pfad.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 15 / 79

Fetch und R-Type-Datenpfad

- Aufbau des Hardware-Schaltplans beginnen
- Fetch: Instruktion holen
- PC und PC+4-Addierer
- Verschaltung für `add/sub`
- Timing innerhalb eines Taktes



Notizen

Der nächste Abschnitt soll add, sub, and und or ausführen und den PC um vier Byte erhöhen.
R-Type, PC und ALU werden dafür verschaltet: zwei Leseports, ALU, Schreibport und PC+4.
Die kleine Kette auf der Folie ist nur die Übersicht. Sie ersetzt den Datenpfad mit zwei Leseports noch nicht.
Beispiel: Die Ports kommen auf den folgenden Folien in fester Reihenfolge dazu.
Zuerst der PC als Adress-Zustandsspeicher.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 16 / 79

Der Startpunkt: Der PC

- Program Counter: spezielles 32-Bit-Register
- Speichert den Ausführungszustand (State)
- D-Flip-Flops: Update nur bei positiver Taktflanke
- Während des Taktes: stabile Adresse am Ausgang

Notizen

Der PC speichert die aktuelle Instruktionsadresse, 32 Bit in diesem Modell. Er ist nicht der ganze Zustand: Register und beschreibbarer Speicher gehören dazu.
Q ist der stabile Ausgang nach der Clock-to-Q-Verzögerung, D der vorbereitete Eingang. Eine positive Flanke übernimmt D nach Q. Stabiler Ausgang heißt nicht, dass D während der Periode unverändert bleiben muss.
Reset des PC ist eine Umgebungsentscheidung, keine ISA-Adresse.
Beispiel: PC_Q=0x100 kann bestehen, während PC_D schon 0x104 zeigt.
PC_Q adressiert als Nächstes den Instruktionsspeicher.

Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 17 / 79

Instruction Memory (Befehlsspeicher)

- Erstes Bauteil hinter dem PC
- PC-Wert am Adress-Eingang
- Single-Cycle-Modell: kombinatorische Ausgabe der Instruktion
- Während der Ausführung: nur Lesen



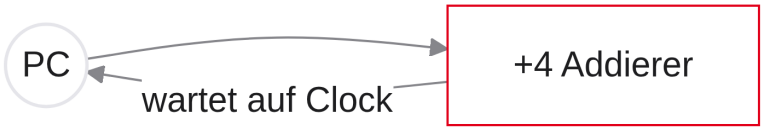
Notizen

Der Instruktionsspeicher hat einen Adressport und einen Instruktionsport. Das Lesen ist im Modell kombinatorisch und ändert das Wort nicht.
Nur-Lesen während der Ausführung ist eine Lehrmodellannahme, kein Satz über jedes Flash oder RAM der Welt.
Ein synchroner Speicher mit Extra-Takt wäre ein anderes Timing.
Beispiel: PC=0x100 liefert genau das dort liegende 32-Bit-Wort an die Decodierung.
Parallel dazu entsteht die sequenzielle Folgeadresse.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 18 / 79

PC-Inkrementer ($PC + 4$)

- Separater Hardware-Addierer (nicht die Haupt-ALU)
- Fest verdrahtet: immer $+4$ zur PC-Adresse
- Ergebnis zurück an den PC-Eingang — wartet auf die nächste Flanke



Notizen

Die Konstante 4 ist der Byteabstand unkomprimierter Instruktionen, nicht vier Takte.
Ein eigener Addierer bildet PC_alt+4, während die Haupt-ALU später Operanden bearbeitet. Vor der Flanke bleibt PC_Q=0x100, der Kandidat ist 0x104. Nach der Flanke ist PC_Q=0x104, sofern kein Branch gewählt wurde.
Das Rückkabel endet am D-Eingang des Flip-Flops. Es gibt keinen kombinatorischen Dauer-Inkrementierer.
Beispiel: Im Gesamtmodell wählt ein MUX später zwischen PC+4 und Branch-Ziel.
Die R-Type-Felder bestimmen jetzt die drei Registerports.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 19 / 79

R-Type-Refresher

- Instruktion liegt auf den Leitungen
- Ziel: `add rd, rs1, rs2` physikalisch
- Bits 19:15 = `rs1`, 24:20 = `rs2`, 11:7 = `rd`

Bits	Feld
31:25	`funct7`
24:20	`rs2` (lesen)
19:15	`rs1` (lesen)
14:12	`funct3`
11:7	`rd` (schreiben)
6:0	`opcode`

Notizen

instr[31] ist das höchstwertige Bit. rs1, rs2 und rd sind je 5 Bit, also 32 mögliche Nummern.
0x007302B3 ist add mit rd=5, rs1=6, rs2=7, opcode=0x33, funct3=0, funct7=0. sub behält opcode und funct3, setzt funct7 aber auf 0x20. Ohne funct7 sind add und sub nicht unterscheidbar.
Die Feldverdrahtung ist keine zweite Assemblierung. Der Assembler ist fertig.
Beispiel: and hat funct3=7, or funct3=6, beide mit funct7=0.
Die zwei Quellnummern führen zu zwei Leseausgängen.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 20 / 79

Register lesen (ID)

- Zwei Lese-Ports A1 und A2
- `rs1`/`rs2` direkt angeschlossen
- Kombinatorisch: Inhalte auf RD1 und RD2
- Operanden physikalisch isoliert



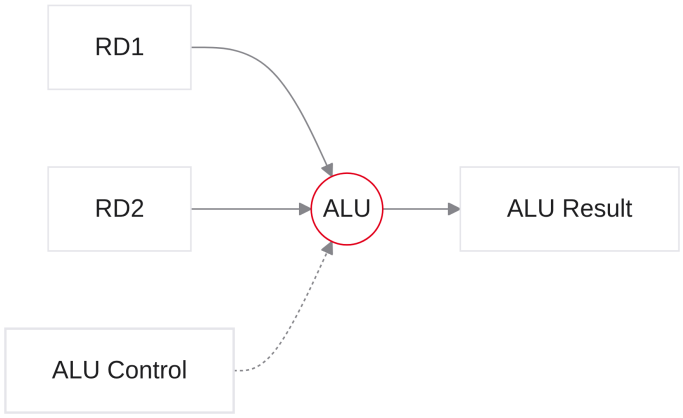
Notizen

ID liest zwei Register gleichzeitig. A1 und A2 sind Adressports, RD1 und RD2 Datenports. Es sind keine zwei getrennten Registerbänke.
A1=6 und A2=7 liefern die gespeicherten Werte, im Beispiel 10 und 15. Dieselbe Nummer an beiden Ports ist erlaubt und liefert denselben alten Wert zweimal.
Die Bank speichert Zustand, die Leseausgabe des Modells ist kombinatorisch. Getrennte Ausgänge heißen nicht physikalisch isolierte Welten.
Beispiel: 5 Bit Adresse und 32 Bit Daten müssen im Bild getrennt sein.
Diese beiden Datenwörter gehen an die ALU.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 21 / 79

Die ALU (EX)

- RD1 und RD2 in die ALU-Eingänge
- Sofortige Verknüpfung (kombinatorisch)
- Operation über ALU-Control aus opcode, `funct3` und bei R-Typ auch `funct7`
- Ergebnis am ALU-Ausgang



Notizen

Die ALU verknüpft RD1 und RD2 kombinatorisch. Die Operation kommt aus der Steuerung und braucht bei add/sub auch funct7, nicht nur opcode und funct3.
Dieselben Operanden 10 und 15 ergeben bei ADD 25 und bei SUB −5 als 32-Bit-Zweierkomplement. and und or sind bitweise und nicht das Steuer-UND vor dem PC.
Sofort heißt nach der Gatterverzögerung, nicht ohne Laufzeit. Das Ergebnis ist noch nicht gespeichert.
Beispiel: 25 liegt am ALU-Ausgang, t0 enthält noch den alten Wert.
Der Ausgang braucht einen Weg zum Schreibport.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 22 / 79

Write-Back (WB)

- Ergebnis zurück zur Werkbank
- Leitung vom ALU-Ausgang zum WD-Port (Write Data)
- Visuell der „rückwärts“-Pfad auf dem Plan
- Bits warten am Register File



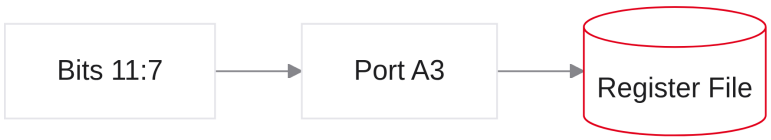
Notizen

WB führt das ALU-Ergebnis zum WD-Port, Write Data. Die Registerbank braucht außerdem die Zielnummer und RegWrite.
25 am WD-Port ist noch kein neuer Inhalt von x5. RD1 und RD2 sind Leseausgänge, WD ist der Schreibeingang.
Die rückwärts gezeichnete Leitung ist ein normaler gerichteter Weg.
Beispiel: Ohne RegWrite bleibt der alte Registerzustand, auch wenn WD schon 25 zeigt.
Die Zielnummer legt fest, welches Register gemeint ist.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 23 / 79

Zieladresse (`rd`)

- Schreibfach: Bits 11:7 (`rd`) an Port A3
- Hardware für R-Type damit vollständig verdrahtet



Notizen

rd steht in instr[11:7] und geht an A3, fünf Bit. A3=5 und WD=25 bedeuten: Register x5 soll den Wert 25 erhalten, nicht dass die Adresse 25 wäre.
Vollständig ist der Weg erst mit RegWrite und der Flanke. Ein Schreibversuch auf x0 ändert die gespeicherte Null nicht.
Adresse und Datenwort sind im Bild getrennt zu beschriften.
Beispiel: t0 ist x5, der Wert 25 ist nicht die Nummer 5.

Eine Zieladresse allein darf noch nicht schreiben.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 24 / 79

Das RegWrite-Signal

- Schutzventil: Steuerpin RegWrite (1 Bit)
- RegWrite $= 1$: bei nächster Flanke in `rd` speichern
- RegWrite $= 0$: Signale prallen ab (wichtig für `sw`/`beq`)

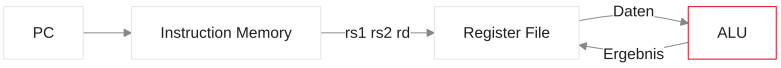
Notizen

RegWrite ist die einbitige Register-Schreibfreigabe. Bei 1 übernimmt das Ziel an der Flanke WD. Bei 0 bleiben die Register alt, obwohl WD und A3 konkrete Werte haben können.
add setzt RegWrite=1, sw und beq setzen 0. Die CPU prüft keinen Algorithmus, sie decodiert Felder.
x0 bleibt auch bei RegWrite=1 null.
Beispiel: Vor der Flanke zeigt WD=25, nach der Flanke enthält t0 die 25.
Datenweg, Ziel und Freigabe ergeben den R-Type-Pfad.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 25 / 79

Der fertige R-Type-Pfad

- Strom: PC → Instruction Memory → Register lesen → ALU → Write-Back
- Kein Zugriff auf den Data Memory



Notizen

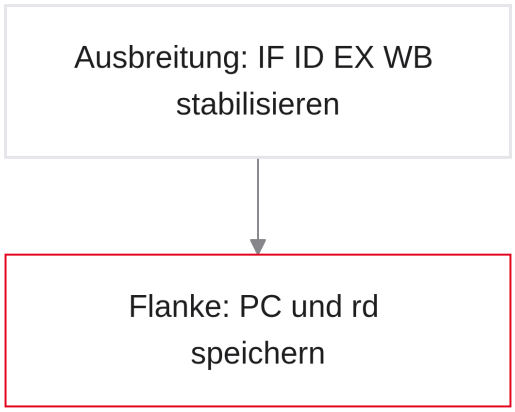
Der R-Type-Pfad verbindet PC, Instruktionsfelder, zwei Leseports, ALU, WD, A3, RegWrite und PC+4.
Beispiel add t0, t1, t2 mit x6=10 und x7=15: vor der Flanke sind RD1=10, RD2=15, ALU=25, RegWrite=1, PC_D=PC+4. Nach der Flanke ist x5=25 und der PC um vier Byte weiter.
Ohne RAM heißt ohne Datenspeicherzugriff, nicht ohne Instruktionsspeicher. Es ist kein Pipelinelauf durch fünf getaktete Kästen.
Beispiel: Zuerst Zustand, dann kombinatorische Werte, zuletzt die Flanke.

Als Nächstes: wann werden diese Werte wirklich Zustand?
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 26 / 79

Timing im Single-Cycle

- Gatter arbeiten ununterbrochen; Signale breiten sich aus
- Erst wenn EX/WB stabil: Ergebnis am Register-Eingang
- Steigende Flanke am Taktende:
  - Register File speichert (wenn RegWrite $= 1$)
  - PC übernimmt $PC + 4$
- Sofort Start des nächsten Befehls



Notizen

Ein Zeitbild braucht zwei Flanken, nicht fünf gleich lange Phasenkästen.
Nach der Startflanke laufen die Verzögerungen. Vor der nächsten Flanke müssen die D-Eingänge die Setup-Zeit erfüllt haben. PC und Register können an derselben Flanke übernehmen. Danach gehören die Leitungen zur nächsten Instruktion.
Zahlen in diesem Bild sind Lehrwerte. Die längere kritische-Pfad-Rechnung folgt später.
Beispiel: PC_Q bleibt zwischen den Flanken der alte Wert, WD darf sich schon auf das neue Ergebnis setzen.
Dieselbe Flanke erlaubt sogar Lesen und Schreiben desselben Registers.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 27 / 79

Warum überschreiben wir nichts vorzeitig?

- Fall: `rd` = `rs1`, z. B. `addi t0, t0, 1`
- Alter Wert wird gelesen, durch ALU, steht am Schreib-Port
- Flip-Flops ändern den Ausgang erst mit der Flanke
- Deshalb ist $a = a + b$ gefahrlos

```
t0 = t0 + t1;
// Lesen und Schreiben im selben Takt – Flankensteuerung rettet uns
```

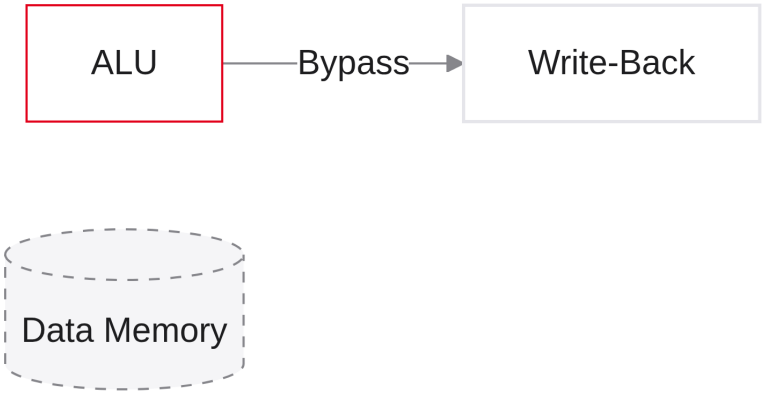
Notizen

Besser als ein unmarkiertes addi ist add t0, t0, t1. Bei t0=10 und t1=15 lesen beide Ports den alten t0 beziehungsweise t1. WD wird 25. Die Flanke schreibt 25. Es entsteht kein kombinatorischer Selbstaddierer, weil der neue Wert erst im Flip-Flop ankommt.
a=a+b meint hier den Registerzustand. Das ist keine Aussage über C-Überlauf. addi wäre eine Erweiterung über den vorhandenen I-Pfad, nicht der Basis-R-Typ.
Alter Q-Wert und neuer D-Wert stehen gleichzeitig im Bild.
Beispiel: Vor der Flanke liefert ein erneutes Lesen noch 10.
Der R-Type nutzt keinen Datenspeicher; der kommt als Nächstes.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 28 / 79

Was passiert mit Phase 4 (MEM)?

- Im reinen R-Type-Pfad fehlt der Data Memory
- MEM existiert logisch, physikalisch Bypass
- ALU-Ergebnis direkt zum Write-Back
- Nächster Schritt: RAM für Load/Store einhängen, ohne R-Type zu zerstören



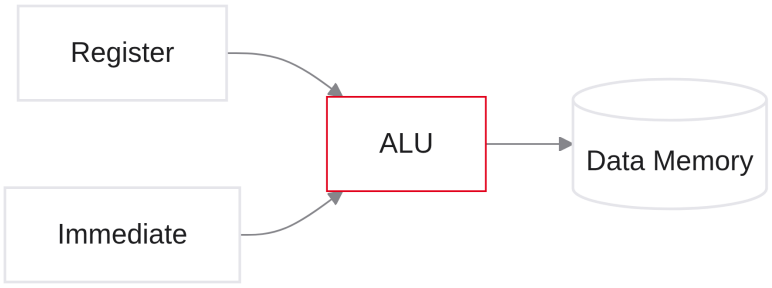
Notizen

MEM kann für add unbenutzt bleiben. Das ALU-Ergebnis geht direkt zum Write-Back. Der Data-Memory-Block darf trotzdem im Prozessor vorhanden sein.
Bypass heißt alternativer Datenweg, keine zusätzliche Taktphase. Nicht angeforderte Leitungen sind nicht automatisch null.
add setzt MemRead und MemWrite auf 0.
Beispiel: 25 geht an WD, nicht an den Speicher-Adressport.
lw und sw brauchen einen Offset statt des zweiten Registeroperanden.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 29 / 79

Der Load/Store-Datenpfad

- Konstanten (Immediates) in die ALU
- Data Memory einbinden
- Erster Ressourcenkonflikt — Multiplexer
- Datenwege für `lw` und `sw`
- Steuersignale MemRead / MemWrite



Notizen

Load liest ein Wort, Store schreibt ein Wort. Der ALU-A-Eingang bleibt die Basis aus RD1. Der zweite Operand kann ein Offset sein. Bei lw und sw ist der ALU-Ausgang eine Adresse. lw holt danach ein anderes Datenwort aus dem Speicher.
Zwei Auswahlstellen entstehen: ALU-B und Register-Write-Back. Ein Konflikt im Entwurf heißt, dass zwei Quellen nicht fest miteinander verbunden werden dürfen. Ein korrekt gebauter Prozessor hat diesen Konflikt nicht mehr.
Beim add ist die zweite Quelle ein Registerwort, beim lw ein Offset.
Beispiel: Die Quellen heißen RD2 und Immediate.
Zuerst wird der Offset aus dem I-Format gewonnen.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 30 / 79

Das Problem mit I-Types

- R-Type: beide Operanden aus dem Register File (`rs1`, `rs2`)
- I-Type (`lw`, `addi`): kein `rs2` — stattdessen 12-Bit-Immediate
- Die ALU hat 32-Bit-Eingänge
- Die Konstante muss aus dem Befehlswort in die ALU gelangen

Bits	Feld
31:20	`imm[11:0]` → diese 12 Bits müssen in die ALU
19:15	`rs1`
14:12	`funct3`
11:7	`rd`
6:0	`opcode`

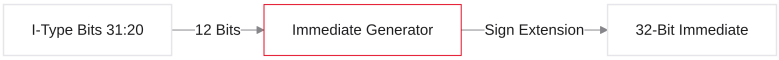
Notizen

Am Beispiel lw t0, 12(s1) steht in den Bits 31:20 ein 12-Bit-Offset und kein rs2 für den zweiten Rechenoperanden. addi hat dasselbe Immediate-Feld, ist aber hier eine Erweiterung, kein Basisbefehl.
Die ALU erwartet 32 Bit. Die oberen Bits dürfen nicht offen bleiben. Die richtige Ergänzung ist Vorzeichenerweiterung, kein offenes Lötfeld.
+12 wird 0x0000000C. −4 hat zwölf Bit 0xFFC und wird 0xFFFFFFFC. Mit Basis 0x1000 entstehen 0x100C beziehungsweise 0x0FFC.
Beispiel: Registerbits und Konstantenbits haben verschiedene Bedeutung.
Ein Immediate Generator baut den 32-Bit-Wert.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 31 / 79

Immediate Generator (ImmGen)

- I-Teil: Bits 31:20, vorzeichenerweitert auf 32 Bit
- Der volle ImmGen liest die zum Format gehörenden Felder, nicht pauschal nur zwölf Eingänge
- S verkettet zwei Felder, B enthält das implizite Bit 0 bereits im Byteoffset



Notizen

ImmGen heißt Immediate Generator. Ein I-Teil liest zwölf zusammenhängende Bits und erweitert sie. Der volle Baustein für I, S und B erhält die passenden, teils verteilten Felder und die Formatauswahl. Er ist nicht generell ein Baustein mit nur zwölf Eingängen.
I: Vorzeichenerweiterung von instr[31:20]. S: Vorzeichenerweiterung der Verkettung instr[31:25] mit instr[11:7]. B: Vorzeichenerweiterung von instr[31], instr[7], instr[30:25], instr[11:8] und einer angehängten Null. Die Klammern sind Verkettung, keine Addition.
Nach dem fertigen B-Byteoffset gibt es kein zweites Shift-left-1. +12 bleibt +12, −4 bleibt −4.
Beispiel: Bit 31 wird beim I-Typ in die oberen 20 Bits kopiert. Bei B ist das nicht dieselbe Regel.
Immediate und RD2 konkurrieren nun um den ALU-B-Eingang.

Folie 32 / 79

Der erste Konflikt (Kollision)

- Zwei 32-Bit-Signale wollen denselben ALU-B-Eingang
- `add`: Signal aus RD2 (Register File)
- `lw`/`addi`: Signal aus ImmGen
- Beide Kabel auf einen Kontakt = Kurzschluss

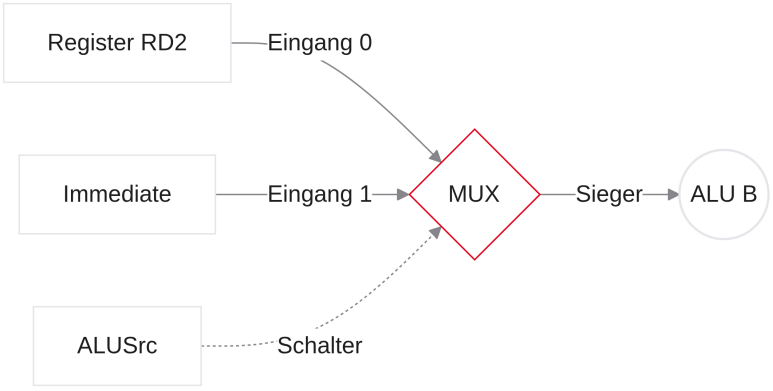
Notizen

RD2 und der ImmGen-Ausgang sind zwei 32-Bit-Quellen für denselben ALU-B-Port.
add braucht das zweite Registerwort, lw den Offset. Eine korrekte Schaltung verbindet die Ausgänge nicht fest miteinander. Zwei unabhängig getriebene Ausgänge an einem Knoten können einen Treiberkonflikt erzeugen. Immer Kurzschluss ist zu grob. Es ist auch kein Pipeline-Hazard.
Beispielwerte: RD2=15 und Immediate=12. Nur einer darf den ALU-Port treiben.
Beispiel: Der nicht gewählte Wert darf auf seiner eigenen Leitung bleiben.
Ein Multiplexer wählt genau eine Quelle.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 33 / 79

MUX 1: ALU-Source (`ALUSrc`)

- Multiplexer: mehrere Eingänge, ein Ausgang
- Steuerpin `ALUSrc`
- `ALUSrc = 0`: RD2 zur ALU
- `ALUSrc = 1`: Immediate zur ALU



Notizen

Ein MUX, Multiplexer, hat zwei 32-Bit-Eingänge, einen 32-Bit-Ausgang und ein getrenntes Select-Bit. ALUSrc heißt ALU Source.
Im Lehrmodell bedeutet 0 RD2 und 1 das Immediate. Das ist eine Entwurfswahl, keine ISA-Regel. Der nicht gewählte Eingang kann einen Wert führen. Der MUX speichert nichts; er ist kombinatorisch.
add mit ALUSrc=0 gibt 15 weiter, lw mit ALUSrc=1 gibt 12 weiter.
Beispiel: Select und Datenwort haben verschiedene Bitbreiten.
Damit kann die ALU für lw die effektive Adresse rechnen.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 34 / 79

Load Word (`lw`) — die Adressierung

- Mit MUX kann die ALU Konstanten verarbeiten
- Bei `lw`: ALU berechnet nur die RAM-Adresse (Base + Offset)
- Ergebnis = Hausnummer, kein „normales“ Rechenergebnis
- Diese Adresse geht an den Datenspeicher

```
lw t0, 12(s1)
# 1. ALUSrc = Immediate (12)
# 2. ALU: s1 + 12
# 3. Adresse am ALU-Ausgang
```

Notizen

lw heißt load word. Die effektive Adresse ist Basis plus Byteoffset.
Lehrwerte: s1=0x1000, Offset 12, ALUSrc=1, ALU-Operation ADD, Adresse 0x100C. t0 ist vor der Flanke noch unverändert. 12 Byte sind drei Wörter, nicht Wortindex 12.
Der Speicherinhalt an 0x100C sei zur Illustration 88. Adresse und Daten sind verschieden. 0x1000 ist keine universelle Ripes-Adresse; das Labor nutzt ein Label.
Beispiel: Wortzugriffe bleiben auf Vielfache von vier ausgerichtet.
Die Adresse ist fertig, der Load-Wert kommt aus dem Datenspeicher.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 35 / 79

Data Memory (Datenspeicher)

- Block in der MEM-Phase: Data Memory (RAM)
- Address: Ausgang der ALU
- Write Data: für Store
- Read Data: gelesene Daten
- Im Modell: kombinatorisches Lesen (wie Instruction Memory)



Notizen

Data Memory hat Address, Write Data und Read Data. Ein Load liest im Modell kombinatorisch. Ein Store schreibt an der Flanke, wenn MemWrite aktiv ist.
Zwei Blöcke bedeuten zwei Zugriffspfade für Instruktion und Daten in derselben Periode. Das sind nicht

zwingend zwei RAM-Chips und kein Beweis getrennter realer Adressräume.
RAM heißt Random Access Memory. Ungültige oder unausgerichtete Adressen sind kein positiver Laborfall.
Beispiel: Adresse 0x100C, Lesedaten 88, Schreibdaten beim Load unbenutzt.
Der Lesewert muss als Write-Back-Quelle zur Registerbank.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 36 / 79

Daten aus dem RAM lesen

- Read Data = Wert des `lw`
- Write-Back: Leitung zurück zum WD-Port des Register Files
- Ziel: bei der Flanke in `rd` speichern
- Kollision: dort liegt bereits das ALU-Rückführkabel



Notizen

Read Data ist das gelesene Wort, WD der Schreibwert der Registerbank, rd die Zielnummer.
Fortsetzung: Adresse 0x100C, Speicherwort 88 oder 0x00000058. An der Flanke soll t0 den Wert 88 erhalten, nicht die Adresse. Der ALU-Ausgang liegt parallel schon an der anderen Write-Back-Quelle. Ohne Auswahl wäre unklar, welche Quelle lw braucht.
Load braucht im Modell keine zweite Flanke für MEM und eine dritte für WB.
Beispiel: Adressleitung und Datenleitung sind im Bild getrennt.
Ein zweiter MUX wählt die Write-Back-Quelle.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 37 / 79

Der zweite Konflikt (`MemtoReg`)

- WD empfängt nur ein 32-Bit-Signal
- R-Type will ALU-Ergebnis; `lw` will RAM-Daten
- Lösung: zweiter MUX vor dem Register File
- Steuersignal: `MemtoReg`

Notizen

MemtoReg heißt Memory to Register. Port 0 ist das ALU-Ergebnis, Port 1 der Speicherlesewert.
add wählt 25, das Beispiel-lw wählt 88. RegWrite ist bei beiden 1. MemtoReg ersetzt weder die Zieladresse noch die Freigabe.
Das Zwei-Wege-Schema ist das Basismodell. Ripes kann zusätzlich PC+4 zurückschreiben. sw braucht gerade kein Register-Write-Back.
Beispiel: Tabelle: add Quelle ALU, lw Quelle Speicher, sw kein Registerschreiben.
Beim Store ändern sich Richtung und Schreibziel.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 38 / 79

Store Word (`sw`) — der Pfad

- `sw`: `rs2` liefert die zu speichernden Daten
- Wert liegt schon auf RD2 (ID-Phase)
- Abzweig von RD2 zum Write-Data-Port des RAM (Bypass um die ALU)
- ALU berechnet parallel Base + Offset als Adresse



Notizen

sw heißt store word, vier Byte, nicht ein Byte. s1 ist die Basis, t0 im Beispiel die Datenquelle, 12 der Byteoffset.
Lehrwerte s1=0x1000 und t0=88: RD1=0x1000, RD2=88, ALUSrc wählt 12, die ALU bildet 0x100C. RD2 zweigt vor dem ALU-MUX zum Write-Data-Port ab. Gespeichert wird 88, nicht 12 und nicht die Adresse.
Das S-Immediate ist die Vorzeichenerweiterung von instr[31:25] verkettet mit instr[11:7]. Mit der Flanke

und MemWrite ändern sich vier Speicherbytes. RegWrite bleibt 0.
Beispiel: Die beiden Pfade laufen gleichzeitig: Adresse über die ALU, Daten über RD2.
Adresse und Daten allein dürfen noch nicht jeden Store auslösen.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 39 / 79

Die Signale `MemWrite` und `MemRead`

- Speicher braucht Schutzpins wie das Register File
- `MemWrite = 1`: bei Flanke schreiben
- `MemRead = 1`: Daten auf Read Data ausgeben
- Bei `add`: beide 0 — RAM inaktiv

Notizen

MemWrite ist die Speicher-Schreibfreigabe, MemRead im Lehrmodell die Kennzeichnung eines Lesezugriffs.
add: beide 0. lw: Read 1, Write 0. sw: Read 0, Write 1. beq: beide 0. Ein Wert am Write-Data-Port ohne MemWrite ändert den Speicher nicht. MemWrite ist kein Don't Care. MemRead=0 heißt nicht, dass das RAM elektrisch aus und alle Ausgänge null sind.
Ripes muss keinen Pin namens MemRead zeigen.
Beispiel: Vor der Flanke ist die Freigabe sichtbar, der Speicherinhalt noch alt.
Der Store muss außerdem das Registerschreiben unterbinden.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 40 / 79

Store und das Register File

- `sw` schreibt in den RAM, nicht in Register
- S-Type hat kein `rd`; dieselben Bits sind Immediate und `rs2`
- `RegWrite` muss 0 sein, sonst wird das aus den Bits decodierte Register beschrieben

```
Ohne RegWrite=0:
MUX MemtoReg leitet immer etwas weiter -
Store wuerde Register mit Muell beschreiben.
```

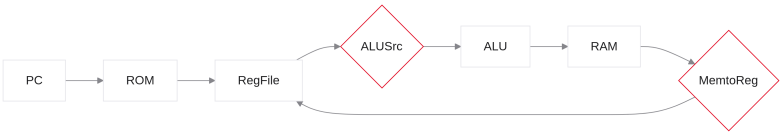
Notizen

Im S-Format ist instr[11:7] imm[4:0], nicht rd. rs2 steht in instr[24:20]. Dieselben Bits sind also nicht Immediate und rs2.
RegWrite bleibt 0. Am WD-Port darf ein konkreter, für den Store irrelevanter Wert liegen; er wird nicht übernommen. MemtoReg darf X sein, weil kein Registerschreiben erfolgt. X ist keine gleichzeitige 0 und 1 und kein offener Draht. RegWrite und MemWrite dürfen nicht X sein.
Vorher: Speicherwort alt, Register unverändert. Nachher: vier Bytes neu, GPR gleich.
Beispiel: Eine feste Verdrahtung von Bits 11:7 als A3 wäre für sw keine gültige Zielangabe.
R-Type, Load und Store lassen sich nun zusammenfassen.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 41 / 79

Der Load/Store-Pfad erweitert

- Rechnen (R-Type) und Speicherzugriff (I-/S-Type)
- Multiplexer regeln den Verkehr
- Es fehlt: dem PC das Springen beibringen



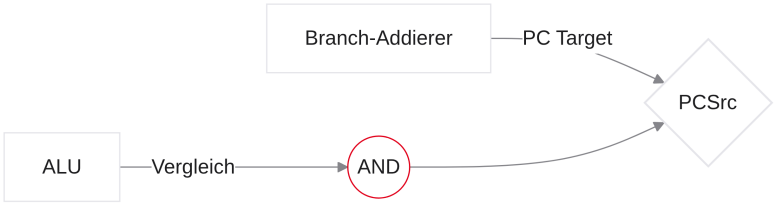
Notizen

Der gemeinsame Pfad hat ALUSrc, MemtoReg und den RD2-Abzweig zum Speicher. Eine reine Kette ALU, Speicher, Register würde den ALU-Bypass und die Storedaten unterschlagen.
add: ALUSrc=0, MemtoReg=0, RegWrite=1. lw: ALUSrc=1, MemtoReg=1, RegWrite=1. sw: ALUSrc=1, RegWrite=0, MemWrite=1, Daten aus RD2.
Bis hier ist PC+4 der einzige ausgeführte nächste PC. Programme bleiben linear.
Beispiel: Dieselben MUX-Codierungen wie zuvor gelten weiter.
Ein bedingter Branch braucht Vergleich und PC-Auswahl.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 42 / 79

Der Branch-Datenpfad

- Hardware für bedingte Entscheidungen
- Problemstellung `beq`
- ALU als Komparator (Zero-Ausgang)
- Zweiter Addierer für das Branch-Target
- Logische Verknüpfung der Sprungbedingung



Notizen

beq stellt zwei Fragen: Sind die Registerwerte gleich, und welches Ziel hätte der Sprung?
Im Lehrmodell vergleicht die Haupt-ALU per Subtraktion, ein eigener Addierer bildet PC_alt plus Byteoffset.
Die Entscheidung ist Branch und Zero, nicht ein AND mit nur einem Eingang.
RegWrite bleibt 0. Ripes darf dieselbe Semantik mit einem eigenen Vergleichsblock lösen. Das ist eine andere Verschaltung, keine andere ISA.
Beispiel: 7 und 7 sind gleich, 7 und 9 nicht. Das Ziel kann in beiden Fällen schon berechnet sein.
Vergleich und Navigation werden am konkreten beq getrennt.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 43 / 79

Das Branch-Problem (`beq`)

- `beq rs1, rs2, Label`: zwei Aufgaben in einem Takt
- Bedingung: sind `rs1` und `rs2` gleich?
- Navigation: neues PC-Ziel = aktueller PC + Offset
- Beides braucht Addition/Subtraktion — nur eine Haupt-ALU

```
# Hardware-Sicht:
# 1. rs1 == rs2 ?
# 2. PC_neu = PC + Offset
# wenn (1): PC = PC_neu
```

Notizen

beq heißt branch if equal. Verglichen werden die Inhalte, nicht die Registernummern. Das Ziel ist PC_alt plus Byteoffset. Bei Ungleichheit folgt PC_alt+4.
beq schreibt kein rd. Dass die Haupt-ALU hier nicht zugleich das Ziel addiert, ist eine Entwurfsentscheidung, kein ISA-Zwang.
Bei 7 und 7 ist die Differenz 0, bei 7 und 9 nicht. Das illustrative Ziel sei in beiden Fällen dasselbe.
Beispiel: Ein normales sub mit Differenz 0 darf nicht springen, solange Branch=0 ist.
Zuerst wird Gleichheit als Differenz null erkannt.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 44 / 79

Vergleich in Hardware (Subtraktion)

- Trick: Subtraktion statt direktem Gleichheitstest
- ALU: Ergebnis $= rs1 - rs2$
- Identisch $\Rightarrow$ Ergebnis $= 0$
- Dann war beq erfolgreich

```
Wenn A == B, dann A - B == 0
```

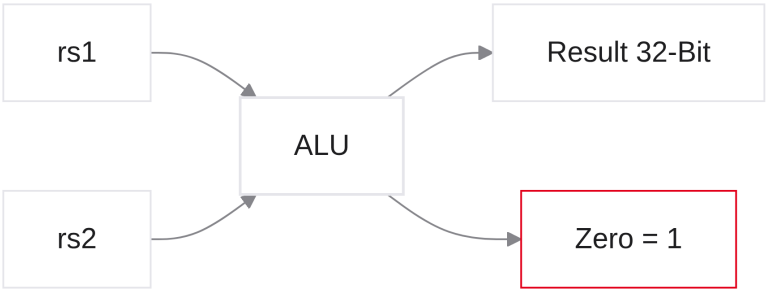
Notizen

7−7 ergibt das Wort 0, 7−9 ein Wort ungleich null. Zwei gleiche 32-Bit-Wörter haben die Differenz null, auch in der modularen Registerarithmetik.
Das Differenzwort ist keine zu speichernde Variable. beq braucht nur die Aussage null oder nicht null. Erfolgreich im Sinne eines Sprungs ist das nur zusammen mit der Befehlsklasse beq.
Die ALU-Operation ist SUB, RegWrite ist 0.
Beispiel: Ein Daten-sub mit gleichen Operanden setzt Zero, aber nicht Branch.
Ein Ein-Bit-Ausgang fasst diese Null-Eigenschaft zusammen.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 45 / 79

Die ALU und der Zero-Ausgang

- Dedizierter 1-Bit-Ausgang: Zero (oft „Zero-Flag“ genannt)
- NOR über alle 32 Ergebnisbits
- Alle Bits 0 $\Rightarrow$ Zero $= 1$
- Sonst Zero $= 0$
- Das Datenwort der ALU wird bei Branches typisch verworfen



Notizen

Zero ist ein kombinatorischer ALU-Ausgang, kein RISC-V-Statusregister. Es ist das negierte ODER aller 32 Ergebnisbits: nur das Nullwort ergibt 1.
0x00000000 ergibt 1, 0x00000001 und 0xFFFFFFFF ergeben 0. Zero ist nicht die Kopie des Datenworts. Die Darstellung gehört zum Lehrmodell. Ripes prüft Branches in einem eigenen Block; dort muss der ALU-Ausgang nicht null sein.
Beispiel: Das Bit gilt für die aktuelle Operation, nicht als gespeichertes Flag.
Parallel wird der Branch-Offset zum Byteabstand.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 46 / 79

Zieladresse berechnen (Navigation)

- Haupt-ALU ist mit der Subtraktion belegt
- Offset steckt im B-Type als zerrissenes Immediate
- ImmGen setzt zusammen und sign-extended auf 32 Bit
- Negative Offsets: Sprünge „nach oben“ (Schleifen)

Bits	Feld
31:25	imm[12] und imm[10:5]
24:20	`rs2`
19:15	`rs1`
14:12	`funct3`
11:7	imm[4:1] und imm[11]
6:0	`opcode`

Notizen

Das B-Immediate setzt sich so zusammen: imm[12]=instr[31], imm[11]=instr[7], imm[10:5]=instr[30:25], imm[4:1]=instr[11:8], imm[0]=0. Das ist Verkettung, nicht Addition der Feldwerte.
Ein positiver Offset +12 und ein negativer Offset −8 sind beide vorzeichenerweiterte Byteabstände. Negativ heißt kleinere Adresse, nicht eine feste Richtung im Editor.
Der Bezug ist die Adresse der Branch-Instruktion, nicht PC+4.
Beispiel: Zweierkomplement braucht kein extra Minus-Kabel.
Der ImmGen-Ausgang muss diesen fertigen Offset liefern.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 47 / 79

Offset-Aufbereitung im Lehrmodell

- ImmGen liefert den vorzeichenerweiterten **Byteoffset**
- Bit 0 ist implizit 0 und bereits enthalten
- Danach kein zweites Shift-left-1
- Der Branch-Addierer rechnet alten PC plus diesen Byteoffset

```
Byteoffset = signextend(B-Immediate mit implizitem Bit 0)
Target     = oldPC + Byteoffset
```

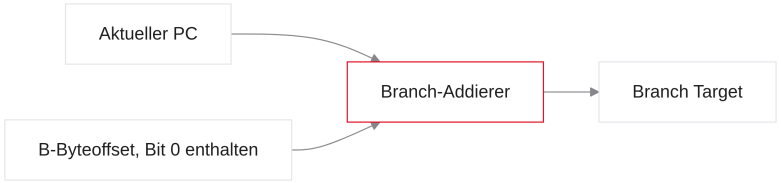
Notizen

Der B-Byteoffset enthält Bit 0 schon als Null. Ein zweites Schieben würde aus +12 die Zahl +24 machen und das Ziel verfehlen.
PC_alt=0x100 plus 12 ergibt 0x10C. PC_alt=0x108 plus −8 ergibt 0x100. Die Codierung erlaubt Zwei-Byte-Schritte; gültige Ziele im unkomprimierten Modell bleiben vier Byte ausgerichtet.
Wortausrichtung und die schon erledigte Skalierung dürfen nicht vermischt werden.
Beispiel: Am Addierer kommt der 32-Bit-Wert +12 an, nicht ein Halbwortoffset.
PC_alt und dieser Offset werden gleichzeitig zur Vergleichsrechnung addiert.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 48 / 79

Der Branch-Addierer

- Zweiter, dedizierter Addierer (Haupt-ALU blockiert)
- Eingang 1: aktueller PC
- Eingang 2: aufbereitetes Offset
- Ausgang: Branch Target



Notizen

Der Branch-Addierer addiert den aktuellen PC_Q und den fertigen Byteoffset. Es gibt keinen Shift-Baustein davor.
Neben der Haupt-ALU existieren der PC+4-Addierer und dieser Zieladdierer. Zweiter Addierer ist nur relativ zu einer benannten Teilmenge sinnvoll.
Während die ALU 7−7 rechnet, kann 0x100+12 schon 0x10C ergeben. Ripes kann die Haupt-ALU für PC plus Immediate nutzen, weil der Vergleich woanders liegt.
Beispiel: Das Ziel ist ein Kandidat, noch nicht der neue PC.
Am PC-Eingang stehen jetzt zwei mögliche Adressen.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 49 / 79

Der dritte Konflikt (PC-Update)

- Am Taktende braucht der PC eine neue Adresse
- Kandidat 1: $PC + 4$
- Kandidat 2: Branch Target
- Zwei Kabel am PC-Eingang — wieder MUX nötig

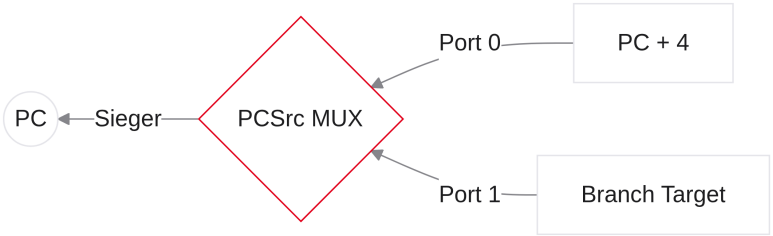
Notizen

PC-Update heißt: an der Abschlussflanke wird eine neue Adresse übernommen.
Bei PC=0x100 können 0x104 und 0x10C beide bereitliegen. Genau eine Quelle bestimmt PC_D. Die andere verschwindet nicht, sie wird nur nicht übernommen.
Gleich oder ungleich schaltet die Zieladdition nicht ab. Gebraucht wird ein MUX, keine zweite PC-Kopie.
Beispiel: Beide Kandidaten sind 32-Bit-Adressen.
PCSrc wird das Auswahlbit.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 50 / 79

MUX 3: PC-Source (`PCSrc`)

- MUX vor dem PC-Eingang
- Port 0: linear ($PC + 4$)
- Port 1: Branch Target
- Steuersignal: `PCSrc`



Notizen

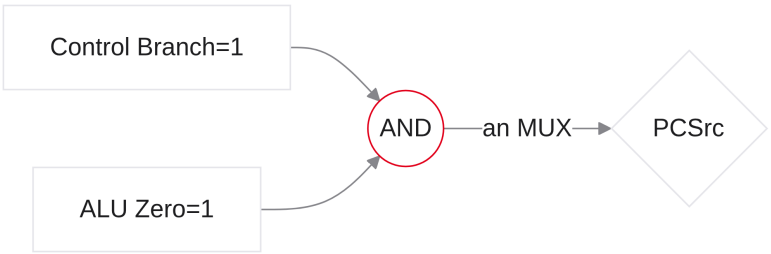
PCSrc wählt Quelle 0 gleich PC_alt+4 oder Quelle 1 gleich Branch Target. Der Ausgang geht an PC_D. Der PC übernimmt an der Flanke.
Die Zeichnung darf nach links zurückführen, die Pfeile müssen die Richtung zeigen. 0 ergibt im Beispiel 0x104, 1 ergibt 0x10C.
PCSrc ist kein Adresswort und ändert keine GPR. Die Codierung 0 und 1 gilt für dieses Lehrmodell.

Beispiel: Der MUX ist kombinatorisch, der PC ist der Zustandsspeicher.
Als Nächstes: wer setzt dieses Bit bei beq?
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 51 / 79

Wer steuert `PCSrc`? (Die Logik)

- Zwei Bedingungen gleichzeitig:
  - Befehl ist Branch (Control: Branch $= 1$)
  - Vergleich erfolgreich (ALU: Zero $= 1$)
- Verknüpfung mit AND-Gatter
- PCSrc $=$ Branch $\wedge$ Zero



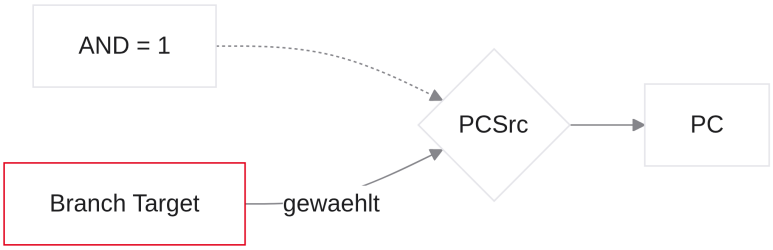
Notizen

Branch ist die Decoderaussage, dass der Befehl beq ist. Zero ist das aktuelle Vergleichsergebnis. PCSrc ist ihr UND, nur in diesem Basismodell.
Die vier Zeilen: 0/0, 0/1 und 1/0 ergeben PCSrc=0, nur 1/1 ergibt 1. Ein sub mit Nullergebnis und Branch=0 bleibt sequenziell. jal und andere Branches brauchen eine andere Entscheidung.
Das UND ist ein Ein-Bit-Gatter, nicht die ALU-Operation and. Der PC speichert den Kandidaten erst an der Flanke.
Beispiel: Bei ungleichem Vergleich bleibt das schon berechnete Ziel ungewählt.
Im Taken-Fall sind Branch und Zero beide eins.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 52 / 79

Trace: Branch Taken

- Bedingung wahr: $\mathtt{beq}$, Register gleich
- AND $= 1$, MUX auf Port 1
- Nächste Flanke: Target in den PC — Sprung erfolgt



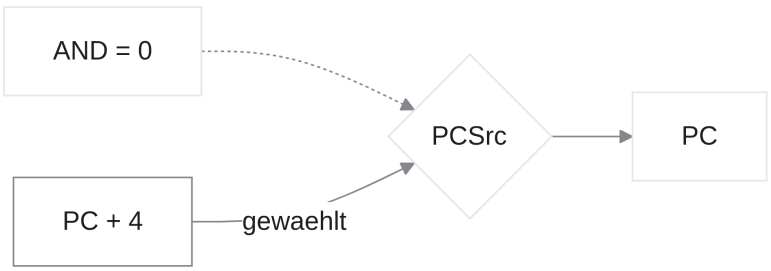
Notizen

Taken heißt: der Sprung wird ausgeführt. Vor der Flanke: PC=0x100, Operanden 7 und 7, Differenz 0, Zero=1, Branch=1, PCSrc=1, Ziel 0x10C, RegWrite=0, MemWrite=0.
An der Flanke wird der PC 0x10C. Register und Datenspeicher bleiben durch beq unverändert. Danach zeigen die Leitungen die Zielinstruktion und nicht mehr die Branch-Werte.
Zwei Zeitpunkte gehören in die Notiz und in das Bild: vor der Flanke und nach der Flanke.
Beispiel: Zero allein ohne Branch würde nicht springen.
Bei ungleichen Werten wählt derselbe Offset die sequenzielle Adresse.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 53 / 79

Trace: Branch Not Taken (Fall-Through)

- Bedingung falsch: Register ungleich
- Zero $= 0 \rightarrow$ AND $= 0$
- MUX bleibt auf Port 0: $PC + 4$
- Target-Adresse verpufft ungenutzt



Notizen

Not Taken heißt nicht genommener Sprung, Fall-through die lineare Fortsetzung. Das ist normale Semantik, kein Fehler.
Bei PC=0x100 und Werten 7 und 9: Differenz ungleich null, Zero=0, Branch=1, PCSrc=0, PC_D=0x104. Das Ziel 0x10C darf parallel vorliegen.
Vor und nach der Flanke ändern GPR und Datenspeicher sich durch dieses beq nicht. Taken und Not Taken sollen gleich aufgebaut nebeneinander stehen.
Beispiel: Die Flanke übernimmt 0x104, nicht 0x10C.
Ein unbedingter Aufruf braucht zusätzlich einen Linkwert.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 54 / 79

Unbedingte Sprünge (`jal`) — Ausblick

- `jal` braucht etwas mehr Verdrahtung als `beq`
- Bedingungslos: `PCSrc` gezielt auf Target
- Rückflugticket: $PC + 4$ muss nach `rd` (meist `ra`)
- Extra-MUX/Kabel zum Write-Back — im Blockdiagramm vereinfacht
- In Ripes im Labor sichtbar

```
jal ra, Label, beide Werte aus dem alten PC:
  nextPC = oldPC + Byteoffset
  ra     = oldPC + 4
```

`jal` liegt außerhalb des Basismodells add/sub/and/or/lw/sw/beq, bis der Write-Back-Pfad für den Linkwert ergänzt ist. Die sequenzielle Schreibweise „erst PC = Target, dann ra = PC+4“ wäre falsch, weil der Linkwert den alten PC braucht.

Notizen

jal liegt außerhalb des Basissubsets. Beide Ergebnisse kommen aus dem alten PC: nextPC = PC_alt + Byteoffset und Linkwert = PC_alt+4.
Beispiel: PC_alt=0x100, Ziel 0x180, ra wird 0x104. Nötig wären J-Immediate, unbedingte Zielwahl und ein Write-Back von PC+4. Branch AND Zero allein steuert jal nicht. Das Zwei-Wege-MemtoReg kann den Linkwert nicht liefern.
ra ist der ABI-Name von x1. Der Pfad kann in Ripes sichtbar sein, gehört aber nicht unmarkiert zum Sieben-Befehle-Modell.
Beispiel: Falsch wäre: erst PC setzen und danach PC+4 als Linkwert lesen.
Die Basisbausteine werden jetzt zusammengefasst.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon

als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 55 / 79

Die Anatomie ist fast komplett

- Speicher/Register: PC, Instruction Memory, Data Memory, Register File
- Rechenwerke: Haupt-ALU, Branch-Addierer, $+4$-Addierer
- Weichen: drei Multiplexer (`ALUSrc`, `MemtoReg`, `PCSrc`)
- Branch-Logik: Zero und AND

Notizen

Zustand haben PC, Registerbank und beschreibbarer Datenspeicher. ALU, Addierer und MUX rechnen kombinatorisch.
Zur Haupt-ALU kommen der PC+4-Addierer und der Branch-Zieladdierer. Drei MUX sind die Auswahlstellen dieses Basismodells, keine universelle Zahl jeder CPU. ImmGen, Branch, Zero und die Schreibfreigaben gehören in die Übersicht.
Ein fehlender ImmGen würde Offsets unsichtbar machen.
Beispiel: RegWrite und MemWrite entscheiden, was an der Flanke Zustand wird.
Die Teilpfade werden in einer Gesamtdarstellung verbunden.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 56 / 79

Der komplette Datenpfad

- Alle Teilpfade zu einem System
- Klassisches Architektur-Diagramm (Patterson & Hennessy)
- Control Unit als Dirigent der Steuersignale
- Schwäche des Single-Cycle: Timing / Taktperiode
- Ausblick: warum Pipelining nötig wird



Notizen

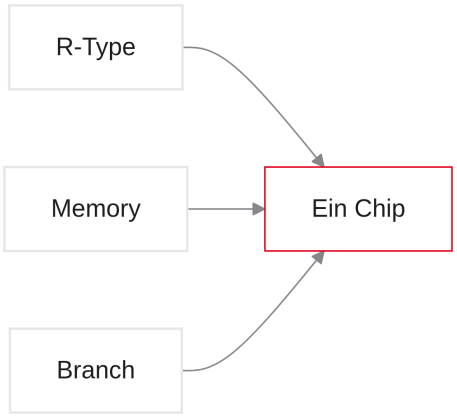
Diese Folie leitet das Gesamtbild ein. Unterstützt sind die sieben Basisbefehle, wenn Datenwege und Steuerung zusammenpassen.
Die Control Unit gehört zum Prozessor; ihr genauer Bau folgt in Einheit 6. Pipelining ist eine Strategie für

höheren Durchsatz, keine ISA-Pflicht und nicht die einzige Alternative.
Ein vollständiges Bild wird danach über einzelne Befehle lesbar gemacht, nicht auswendig gelernt.
Beispiel: Ohne Steuerung wären die MUX unbestimmt.
Warum können verschiedene Befehle dieselben Bausteine nutzen?
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 57 / 79

Der Ressourcenkonflikt (Zusammenfassung)

- Drei Teilpfade: R-Type, Load/Store, Branch
- Problem: keine eigene ALU/Register File pro Befehlstyp
- Transistoren kosten Fläche, Energie und Geld
- Lösung: Pfade überlagern — gemeinsame Nutzung (Reuse)
- Dieselbe ALU: mal Addition, mal Adresse, mal Vergleich



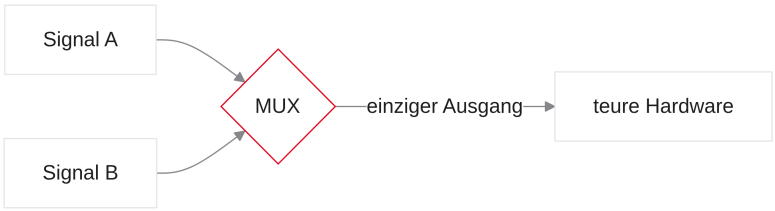
Notizen

Dieselbe Haupt-ALU rechnet beim add Daten, beim lw oder sw eine Adresse und beim beq im Lehrmodell eine Differenz. Das sind zeitlich verschiedene Instruktionen.
Zwei unabhängige Rechnungen in derselben Instruktion brauchen einen weiteren Baustein. Drei ALUs wären nicht absurd, sondern eine andere Abwägung. Dieses Modell wählt gemeinsame Nutzung und Multiplexer. Konkrete Preise oder Energiemengen werden nicht behauptet.
Der Vergleich und die Zieladdition laufen beim beq parallel, deshalb gibt es den Extra-Addierer.
Beispiel: Ein MUX wählt, welche Rolle die ALU gerade hat.
Gemeinsame Eingänge brauchen eine definierte Auswahl.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 58 / 79

Multiplexer: Die Weichensteller

- Wie vermeiden wir Chaos beim Zusammenlegen der Kabel?
- Antwort: unsere drei Multiplexer
- Verkehrspolizisten des Datenpfads
- Ohne sie: Kurzschlüsse an teuren Eingängen (ALU, PC, Register)



Notizen

Die drei Weichen: ALUSrc wählt RD2 oder Immediate, MemtoReg wählt ALU-Wert oder Speicherwort, PCSrc wählt PC+4 oder Branch-Ziel.
Gleiche Breite bedeutet nicht gleiche Bedeutung. Ein MUX ist eine kombinatorische Auswahl. Der ungewählte Eingang muss nicht null sein. Die Stellung steht vor der Flanke fest.
Kurzschlussmetaphern ersetzen die Auswahl nicht: genau eine Quelle wird weitergegeben.
Beispiel: Für add gilt 0, 0 und 0 an den drei Select-Bits im Lehrmodell, mit Branch=0.
Das Gesamtbild muss jede Auswahl an der richtigen Stelle zeigen.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 59 / 79

Der komplette Datenpfad (Patterson & Hennessy)

- Single-Cycle Datapath nach **Patterson und Hennessy** (RISC-V Edition; Signalnamen dieses Kurses)
- Harris & Harris (Kursliteratur) nutzen teils andere Namen (`ResultSrc` statt `MemtoReg`, `ImmSrc`, …) — Konzept identisch
- Vereint IF, ID, EX, MEM, WB; drei MUX-Weichen hervorgehoben
- Rote gestrichelte Linien: Steuersignale der Control Unit

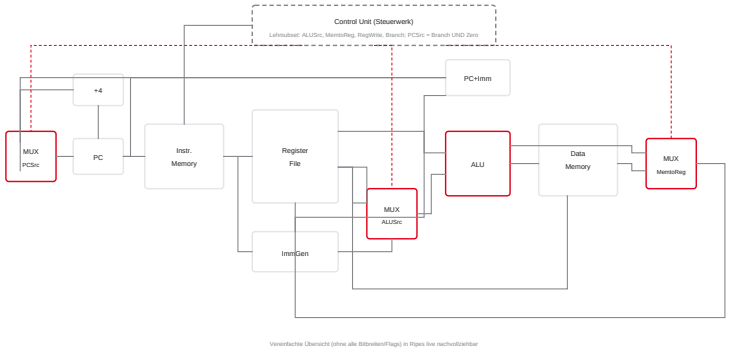


Abbildung 1: Single-Cycle-Datenpfad (vereinfacht)

Notizen

Das zentrale Diagramm ist das Lehrmodell für die sieben Befehle, angelehnt an übliche Lehrbuchzeichnungen und als eigene Darstellung zu lesen, nicht als identische Abbildung einer bestimmten Seitenzahl. Sichtbar sein müssen zwei Leseports, Write-Back mit rd, WD und RegWrite, I/S/B-ImmGen, ALUSrc, der Store-Abzweig, MemtoReg, PC+4, der Zieladdierer, Zero, das Branch-UND und der PC-MUX. PCSrc kommt aus Branch und Zero, nicht als bloßer Opcode-Draht.
add, lw, sw und beq lassen sich auf diesem Bild verfolgen. Detailausschnitte sind besser als unlesbare Miniaturen.
Beispiel: Datenleitungen und Steuerleitungen sind unterschiedlich gekennzeichnet.
Ripes setzt dieselbe Semantik teilweise anders um; das Mapping kommt davor, nicht erst im Labor.

Folie 60 / 79

P&H-Diagramm und Ripes: Mapping

Das Lehrbuch-Diagramm und der Ripes-Single-Cycle-Prozessor sind **nicht baugleich**. Merken Sie sich die Entsprechungen:

Patterson/Hennessy (Folien)	Ripes (Single-Cycle)
ALU-Zero + AND mit `Branch` → `PCSrc`	Dedizierte **Branch-Vergleichseinheit** (alle Branch-Typen)
Separater Branch-Addierer	Branch-Ziel oft über **Haupt-ALU** (PC als Op1)
2-Wege-MUX `MemtoReg`	Oft **3-Wege**-Write-Back-MUX (`Reg wr src`: ALU / Mem / PC+4)
Signalnamen `MemtoReg`, `MemRead`, …	teils andere Bezeichner (Tooltips lesen!)
Farben: beliebig	1-Bit-Signale: **grün = 1**, **grau = 0** (kein „rot = 0“, kein „ausgegraut = X“)

Don’t Care (X) ist ein **Entwurfskonzept** der Wahrheitstabelle — der Simulator zeigt immer einen konkreten Bitwert.

Notizen

P&H steht für Patterson und Hennessy, H&H für Harris und Harris. Das sind Lehrbuchtraditionen, keine Bausteine. Das Ripes-Single-Cycle-Modell ist nicht baugleich.
Im Lehrmodell vergleicht die Haupt-ALU, und ein Addierer bildet das Ziel. In Ripes prüft oft ein eigener Block die Bedingung, und die Haupt-ALU bildet PC plus Immediate. Op2 entspricht funktional ALUSrc, Op1 kann PC oder Register wählen. Der Write-Back-Mux kann ALU, Speicher und PC+4 haben. MemtoReg-Codes dürfen nicht ungeprüft übernommen werden. MemRead muss nicht als Pin erscheinen.
Extended zeigt Steuerkomponenten. Ein grünes Ein-Bit-Signal ist high, ein grün aufleuchtendes Datenwort ist nicht die Zahl 1. X aus der Tabelle ist keine Farbe.
Beispiel: Namen gelten für die konkret geprüfte Fassung; master-Links allein legen die Kursinstallation nicht fest.
Die nächsten drei Folien wiederholen die Lehrmodellfunktion und ordnen Ripes daneben zu.

Folie 61 / 79

Weiche 1: `ALUSrc`

- Vor der Haupt-ALU
- Entscheidung: womit rechnen?
- 0: zweites Register (RD2)
- 1: Immediate aus ImmGen

```
add  t0, t1, t2  -> ALUSrc = 0
addi t0, t1, 5   -> ALUSrc = 1
```

Notizen

ALUSrc=0 wählt im Lehrmodell RD2, ALUSrc=1 das Immediate. add und der Vergleich bei beq nutzen Register, lw und sw das Immediate.
addi bleibt eine markierte Erweiterung. In Ripes kann beq die Haupt-ALU mit PC und Immediate speisen. Die beq-Zeile des Lehrmodells ist deshalb kein garantierter Ripes-Portwert.
Für lw ist die Funktion sicher: der zweite Operand ist der Offset. Der sichtbare Name kann Op2 heißen.
Beispiel: Beim add mit Werten 10 und 15 bleibt ALUSrc auf der Registerquelle.
Die nächste Weiche wählt die Write-Back-Quelle, nicht den Rechenoperanden.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 62 / 79

Weiche 2: `MemtoReg`

- Rechts vor dem Write-Back
- Entscheidung: was nach `rd`?
- 0: ALU-Ergebnis
- 1: RAM-Daten
- Bei `sw`/`beq`: Stellung oft Don't Care (weil `RegWrite = 0`)

```
add t0, t1, t2  -> MemtoReg = 0
lw  t0, 0(s1)   -> MemtoReg = 1
```

Notizen

MemtoReg=0 wählt den ALU-Wert, MemtoReg=1 das Speicherwort. add schreibt 25, lw schreibt 88. sw und beq schreiben nicht; dann ist MemtoReg X.
X heißt: beide Stellungen lassen rd unverändert, solange RegWrite=0. Es ist nicht gleichzeitig 0 und 1. Für lw wäre die falsche Stellung folgenschwer: dann käme die Adresse statt 88 ins Register.

Ripes hat oft eine dritte Quelle PC+4. Deren Codierung wird nur nach Versionsprüfung übernommen.
Beispiel: Vor der Flanke liegt 88 am WD-Port, t0 ist noch alt.
PCSrc bestimmt die nächste Adresse, nicht den Rückgabewert.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 63 / 79

Weiche 3: `PCSrc`

- Oben links vor dem PC
- 0: $PC + 4$ (Fall-Through)
- 1: Branch Target
- Einzige Weiche, die auch vom ALU-Zero abhängt (AND mit Branch)

```
add ...              -> PCSrc = 0
beq ... (genommen)   -> PCSrc = 1
```

Notizen

PCSrc=0 wählt PC_alt+4, PCSrc=1 das Branch-Ziel. Im Basismodell ist PCSrc = Branch UND Zero.
add, lw und sw bleiben sequenziell. beq springt nur bei Gleichheit. Ein sub mit Zero=1 und Branch=0 springt nicht. Die Übernahme erfolgt an der Flanke.
jal und weitere Brancharten brauchen mehr Entscheidungslogik. Nicht die einzige Weiche hängt an Zero; ALUSrc und MemtoReg sind unabhängig.
Beispiel: Bei PC=0x100 und Gleichheit wird 0x10C übernommen, bei Ungleichheit 0x104.
Diese Signale erzeugt die Decodierung der aktuellen Instruktion.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

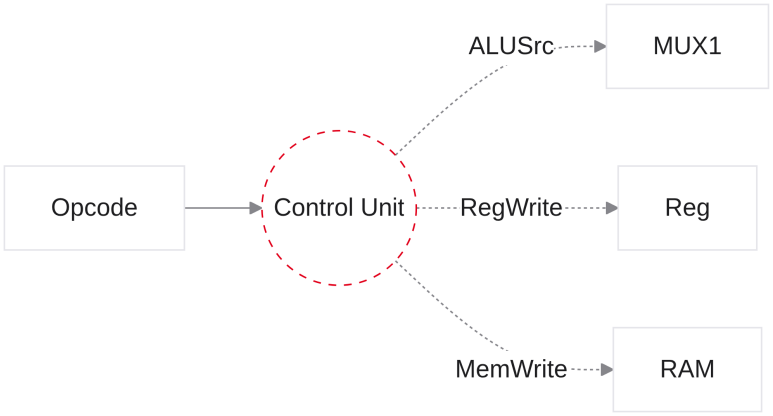
Folie 64 / 79

Die Steuersignale (Control)

- Typisch: RegWrite, MemRead, MemWrite, ALUSrc, ALUOp, …
- Quelle: Control Unit (Steuerwerk)
- Liest den Opcode und erzeugt die Weichen-/Speicher-Signale
- Detailbau der Control Unit: Einheit 6

Befehl	ALUSrc	Mem-toReg	RegWrite	Mem-Read	MemWrite	Branch	ALU	PCSrc
add	0	0	1	0	0	0	ADD	0
sub	0	0	1	0	0	0	SUB	0
and	0	0	1	0	0	0	AND	0
or	0	0	1	0	0	0	OR	0
lw	1	1	1	1	0	0	ADD	0
sw	1	X	0	0	1	0	ADD	0
beq	0	X	0	0	0	1	SUB	Zero

X ist eine für das Ergebnis unerhebliche Auswahl, kein unbekannter Draht. Bei `beq` ist PCSrc gleich Zero. Bei `sub` mit Ergebnis null bleibt PCSrc 0, weil Branch 0 ist.



Notizen

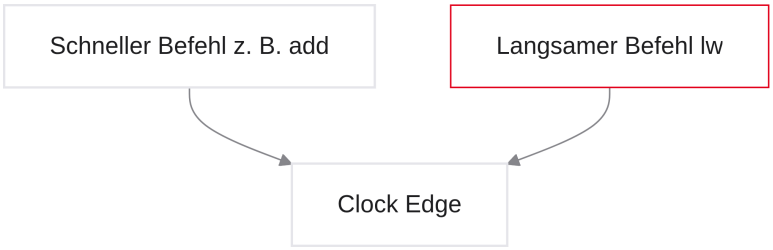
RegWrite, MemRead, MemWrite, ALUSrc, Branch und die ALU-Operation sind die Lehrmodell-Signale. ALUOp und ALUControl sind interne Codierungen, keine ISA-Leitungen. Die konkrete Bitcodierung folgt in Einheit 6.
opcode gibt die Klasse, funct3 und funct7 die ALU-Funktion. PCSrc ist Branch und Zero, nicht der Opcode allein.
Die Tabelle auf der Folie gilt für das Basissubset. Kombinatorische Verzögerung ist keine gemessene Nanosekunden-Zahl einer Technologie.
Beispiel: Ripes-Namen stehen neben der Funktion, nicht statt ihrer.
Der längste nötige Weg begrenzt den Abstand zur nächsten Flanke.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register,

Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 65 / 79

Das Timing-Problem (Critical Path)

- Single-Cycle: alles in einem Takt
- Nächste Flanke erst, wenn das letzte Signal stabil ist
- Taktperiode $\geq$ kritischer Pfad (längste Verzögerungskette)
- Zu kurzer Takt kann einen noch nicht stabilen Wert übernehmen; das ist kein garantierter Absturz durch „Fetch vor Write-Back“



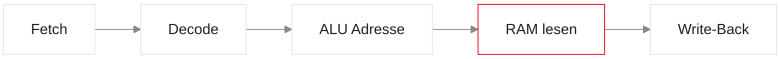
Notizen

Der kritische Pfad ist der zeitbestimmende Weg bis zur nächsten korrekten Zustandsübernahme.
Eine vereinfachte Bedingung ist Taktperiode mindestens Clock-to-Q plus längste kombinatorische Verzögerung plus Setup-Zeit. Parallele Zweige werden mit dem späteren Eingang zusammengeführt, nicht addiert. Clock-Skew ist hier nicht erfunden.
Ein zu kurzer Takt kann einen instabilen Wert übernehmen. Das ist nicht automatisch ein Absturz, weil Fetch vor Write-Back käme. Ripes-Clock misst keine Gatterzeiten.
Beispiel: Die Zahlen der nächsten Folie sind synthetische Lehrwerte.
Welche Instruktion braucht unter diesen Annahmen die längste Periode?
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 66 / 79

Welcher Befehl ist der langsamste?

- Lehrwerte, keine Messung: add 5,5 ns, beq 5,7 ns, lw 7,3 ns
- Gemeinsame Periode mindestens 7,3 ns, idealisiert etwa 137 MHz
- Ripes misst diese Zeiten nicht
- Verlierer: `lw` — alle Stationen inkl. Data-Memory
- Gesamte CPU muss so langsam takten, dass `lw` in einen Zyklus passt



Notizen

Unter den angegebenen Lehrverzögerungen braucht add 5,5 ns bis zur Registerübernahme, beq 5,7 ns bis zur PC-Übernahme und lw 7,3 ns. Die ideale Grenze 1/7,3 ns liegt bei etwa 137 MHz.
Beim beq laufen Zieladdition und Vergleich parallel. Beim lw liegt der ALU-Mux im B-Zweig; die Basis auf A kann später ankommen als das Immediate. Die Zeiten sind keine Ripes-Messung und kein Datenblatt.
Jede Instruktion braucht im Single-Cycle trotzdem einen Zyklus derselben Periode. CPI, cycles per instruction, ist hier 1, die Periode ist lang.
Beispiel: Ein Store ohne Extra-Schreiblatenz bleibt in diesem Zahlenbeispiel unter 7,3 ns.
Dieselbe lange Periode gilt auch für Befehle, die früher fertig wären.

Folie 67 / 79

Das Ende des Single-Cycle

- Langer Takt für `lw` verschwendet Zeit bei schnellen Befehlen
- `add` ist früh fertig — ALU wartet bis zum Tick
- Ineffiziente Hardware-Nutzung
- Für hohe GHz: Architektur umbauen

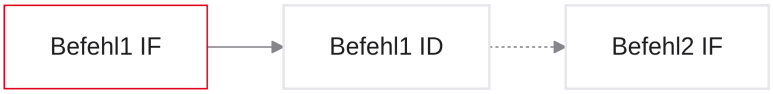
Notizen

Die Grenze des Modells ist die gemeinsame Periode: ein add kann früher stabil sein und wird trotzdem erst an der lw-bestimmten Flanke übernommen.
Daraus folgt eine begrenzte Instruktionsrate, nicht dass alle Transistoren stillstehen oder Single-Cycle unbrauchbar wäre. Das Modell ist einfach und didaktisch klar. Multi-Cycle und Pipeline sind Alternativen. Gigahertz ist kein automatisches Gütemaß. 137 MHz gehören nur zum Lehrbeispiel.
Beispiel: Der add-Pfad ist kürzer, der Takt richtet sich nach lw.
Eine Pipeline kann Tätigkeiten verschiedener Instruktionen überlappen.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 68 / 79

Ausblick: Pipelining (Fließband)

- Langen Zyklus in fünf kurze Takte zerlegen
- Fließband: während Befehl 1 in ID ist, startet Befehl 2 in IF
- Alle Blöcke arbeiten gleichzeitig an verschiedenen Befehlen
- Vertiefung: Einheit 7



Notizen

Die fünf logischen Tätigkeiten können durch Pipeline-Register zu Stufen werden. Die Instruktion zerfällt dabei nicht in fünf Befehle.
Während eine Instruktion in ID ist, kann die nächste in IF sein. Durchsatz und Latenz sind verschieden. Ein Faktor fünf ist kein automatischer Gewinn.
Stufenverzögerung, Registeraufwand und Konflikte folgen in Einheit 7. Heute bleibt das Labor beim Single-Cycle.
Beispiel: Ein kleines Zeitbild zeigt überlappende Instruktionen, keine fünf Takte der alten Periode.
Im Labor wird der Single-Cycle-Prozessor gewählt und die Funktion geprüft.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 69 / 79

Praxis: Ripes Onboarding

- Von abstrakten Diagrammen zur Simulation
- Tool: Ripes — Code-Editor plus Live-Hardware
- Stromsignale auf Leitungen ablesen
- Step-/Clock-Debugging auf den Gattern

Notizen

Ripes ist ein funktionaler RISC-V-Simulator. Onboarding ist der Einstieg in die Bedienung. Clock schaltet den Zustand fort, Tracing protokolliert beobachtete Werte.
Zu sehen sind digitale Portwerte, keine Ströme auf realen Gattern und keine Gatterlaufzeiten. Version und Installationsweg gelten für die im Kurs verwendete Fassung, nicht für jeden master-Stand.
Das Bild ist nicht exakt die Lehrzeichnung. Die Labore liegen als Dateien mit erwarteten Werten vor.
Beispiel: Ein Clock-Schritt ist eine Zustandsfortschaltung, kein Zeitmessgerät.
Venus und Ripes werden nach der jeweiligen Frage eingeordnet.

Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 70 / 79

Venus vs. Ripes

- Venus: Software-Fokus, Algorithmen, Register/RAM-Ergebnisse
- Ripes: Hardware-Fokus, Bits auf Leitungen, MUX und ALU live
- Code in Ripes etwas unbequemer — visuelles Feedback unschlagbar

```
Venus = Blick des Programmierers
Ripes = Blick des Chip-Designers
```

Notizen

Venus eignet sich für die beobachtbare Instruktions-, Register- und Speicherseite. Ripes ergänzt eine konkrete simulierte Mikroarchitektur. Beide sind Software.
Dieselbe Assembleridee kann übertragbar sein. Direktiven, Erweiterungen, Einstieg und ecall-Verträge können abweichen. Der Exit dieser Einheit nutzt in der Ripes-Konvention a7=10, nicht den Venus-Vertrag a0=10 aus Einheit 4.
Unbelegte Urteile über Bedienung werden nicht benötigt.
Beispiel: Die Übungsdateien nennen die Umgebung im Kopfkommentar.
Für den Datenpfad müssen Prozessor und Layout passen.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 71 / 79

Die UI: Prozessorwahl

- Zuerst den richtigen Chip einbauen
- Ripes bietet Single-Cycle und Pipeline-Varianten
- Heute zwingend: Single-Cycle Processor
- 5-Stage Pipeline wäre für heute irreführend

Notizen

Gewählt wird ein 32-Bit-Single-Cycle-Prozessor. Das Extended-Layout zeigt Steuerkomponenten und Bitbreiten; Standard genügt dafür nicht immer. Die Lage von Knöpfen kann sich zwischen Versionen

ändern.
Die C-Erweiterung bleibt aus, damit die betrachteten Instruktionen 4 Byte lang sind. Das Disassembly prüft das. Pipeline oder reiner ISA-Interpreter ersetzen die heutige Ansicht nicht.
UI heißt User Interface. Ein Screenshot wird nur verwendet, wenn er aus der geprüften Version stammt. Hier ist die Anleitung schematisch, weil kein Lauf ausgeführt wurde.
Beispiel: Reset setzt den Simulationszustand zurück.
Danach wird ein kleines Programm geladen.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 72 / 79

Die UI: Editor-Tab

- Assembler wie in Venus (RISC-V)
- Kleine Tests: `add`, `lw`, `beq`
- Ripes assembliert oft live (kein separater Compile-Klick)

```
.text
    li t0, 10
    li t1, 15
    add t2, t0, t1
```

Notizen

.text ist eine Direktive, keine Instruktion. li ist eine Pseudoinstruktion und bei kleinen Zahlen oft ein addi. Dieses Setup liegt außerhalb der sieben Basisbefehle, der gewählte Ripes-Prozessor unterstützt es.
10 und 15 werden zu 25. Im Disassembly ist zu prüfen, welche echte Instruktion gerade aktiv ist. Ein Haltepunkt oder ein Exit verhindert unklaren Fall-through.
Live ohne eigenen Compile-Klick ist eine mögliche Bedienung, keine Garantie jeder Version.
Beispiel: Die Datei beispiele/mct1_vl5_rtype_trace.s enthält den vollständigen Ablauf bis add a0, t0, t1.
Die aktuelle Instruktion wird im Processor-Reiter betrachtet.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 73 / 79

Die UI: Datenpfad-Reiter

- Herzstück: Ripes-Single-Cycle-Implementierung, zugeordnet zum Lehrmodell
- Dicke Linien: 32-Bit-Daten
- Tracing: Clock/Step — einen Takt nach dem anderen
- Run vermeiden, wenn Sie Signale verstehen wollen

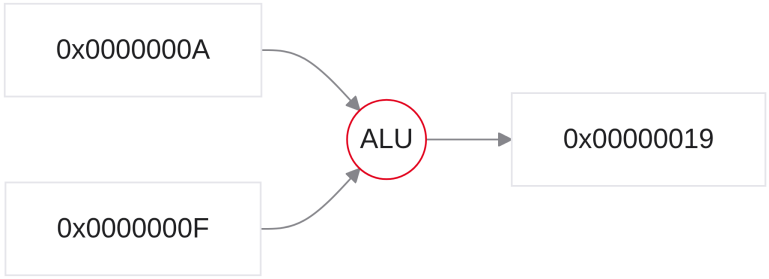
Notizen

Der Reiter zeigt die Ripes-Implementierung, zugeordnet zur Lehrzeichnung, nicht das Lehrbuchdiagramm selbst.
Dicke Linien sind nicht ungeprüft alle 32 Bit; Tooltip oder Beschriftung nennt die Breite. Vor Clock werden PC, Befehl und vorbereitete Werte notiert. Nach Clock werden Register, Speicher und PC geprüft. Die Grafik kann danach schon die nächste Instruktion zeigen.
Clock ist für die erste Beobachtung geeigneter als ein freies Run. Es gibt kein analoges Echtzeitbild.
Beispiel: PC und disassemblierter Befehl müssen zusammenpassen.
Zuerst werden die Datenwerte und ihre Darstellung gelesen.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 74 / 79

Signale live ablesen

- Hex-Werte auf den Leitungen sichtbar
- Datenleitungen vs. 1-Bit-Steuersignale
- Beispiel: $t0 = 10, t1 = 15 \rightarrow$ ALU-Ausgang $0x19$ (25)



Notizen

10 ist 0x0A, 15 ist 0x0F, die Summe 25 ist 0x19. Führende Nullen gehören zum 32-Bit-Wort.
Ein Datenwort und ein Select-Bit sind verschieden. Der angezeigte Wert ist funktional simuliert, keine Messung auf einem Chip. Die Basis der Anzeige ist versionsabhängig und nicht immer dezimal.

Beobachten, während add aktuell ist. Den neuen Registerwert erst nach Clock erwarten.
Beispiel: 0x00000019 am ALU-Ausgang ist 25, nicht ein Ein-Bit-Signal.
Zusätzlich müssen die ausgewählten MUX-Quellen identifiziert werden.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 75 / 79

Multiplexer in Ripes untersuchen

- Bei `lw`: die Auswahl des zweiten ALU-Operanden der Ripes-Version prüfen
- Erwartung der Funktion: Immediate, nicht das zweite Registerwort
- Der sichtbare Portname muss nicht `ALUSrc` heißen

Notizen

Gesucht wird die funktionale Op2-Auswahl der verwendeten Ripes-Version, nicht blind ein Draht namens ALUSrc mit dem Wert 1.
Beim lw ist der zweite ALU-Operand das Immediate. Der erste bleibt für den Load die Registerbasis. Bei einem Branch kann Ripes den PC an die Haupt-ALU legen. Grün an einem mehrbitigen Wert bedeutet Änderung oder Hervorhebung, nicht die Zahl 1.
Vor Clock: beide Operanden, gewählter Eingang, Adresse. Die Codierung des Select-Bits ist versionsabhängig.
Beispiel: Für lw a0, 4(s1) ist der Offset 4, nicht der Speicherinhalt.
Der Store zeigt danach eine Speicheränderung ohne Register-Write-Back.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 76 / 79

Register und RAM inspizieren

- Seitenleisten: Registers und Memory (wie Venus)
- Bei `sw` vor Clock Adresse, Schreibdaten und Freigabe prüfen; nach Clock das Wort, vier Bytes

```
Beobachtung fuer sw a0, 0(s1), Basis B, Daten 88:
1. Vor Clock: aktuelle Instruktion, Adresse B, Write Data 88, Schreibfreigabe
2. Clock ausfuehren
3. Danach: Wort an B ist 88; das Diagramm kann schon die naechste Instruktion
```

Notizen

Vor Clock bereitet sw die Änderung vor. Nach Clock zeigt der Speicher den neuen Zustand.
Im Labor hat lw zuvor 88 nach a0 geschrieben. s1 enthält die Basis B. Bei sw a0, 0(s1) sind Adresse B, Schreibdaten 88 und die Speicherfreigabe aktiv, die Registerfreigabe nicht. Die Portnamen können von MemWrite und RegWrite abweichen.
Nach Clock steht an B das Wort 88 statt 99, an B+4 bleibt 88, a0 bleibt 88. Geschrieben werden vier Bytes.
Danach kann die Grafik schon die nächste Instruktion zeigen, die Freigabe also wieder 0 sein.
Beispiel: Little Endian: 88 erscheint byteweise als 58 00 00 00.
Mit dieser Reihenfolge folgen die eigenen Traces.

Folie 77 / 79

Labor-Aufgabe 1: R-Type-Trace

- Code: `li t0, 50/li t1, 100/add a0, t0, t1`
- Bis zum `add` steppen
- Fragen: Hex auf RD1? Wert von `ALUSrc`? Wert von `RegWrite`?

Notizen

Die Datei beispiele/mct1_vl5_rtype_trace.s hält vor add a0, t0, t1 an, mit t0=50 und t1=100.
Vor Clock: x5=0x32, x6=0x64, rd=x10, ALUSrc=0, MemtoReg=0, RegWrite=1, MemWrite=0, Branch=0, Ergebnis 0x96. Nach Clock: a0=150 und PC bei after_add. Die Codierung des add ist 0x00628533.
li kann zu addi werden. Die Schrittzahl wird am Disassembly geprüft, nicht an Textzeilen. Der Exit nutzt a7=10 und gehört nicht zum Basissubset.
Beispiel: Lehrmodellspalte und Ripes-Entsprechung bleiben getrennt. Diese Erwartung ist statisch geprüft.
Die Load-Aufgabe wechselt die Write-Back-Quelle.
Die Werte auf den Leitungen sind vor der Flanke kombinatorische Ergebnisse des alten Zustands. Register, Speicher und PC übernehmen nur an der Flanke und nur mit der passenden Freigabe. Ein Select-Bit wählt eine Quelle, es ist selbst kein Datenwort. Kontrollfrage: Darf ein am Schreibport anliegender Wert schon als neuer Registerinhalt gelten? Nein, erst nach der Flanke und nur bei aktiver Freigabe.

Folie 78 / 79

Labor-Aufgabe 2: Der Load-Trace

- `.data` mit `.word 99, 88`
- Base in `s1`, dann `lw a0, 4(s1)`
- ImmGen: wird 4 zu 32-Bit?
- Welche Adresse berechnet die ALU?
- Stellung von `MemtoReg` ganz rechts?

```
.data
werte: .word 99, 88
.text
    la s1, werte
    lw a0, 4(s1)
```

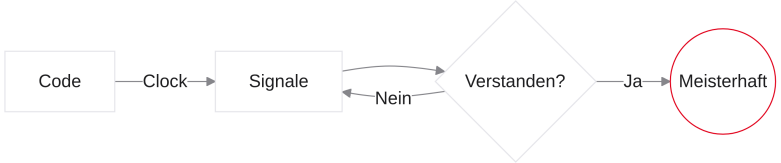
Notizen

.data und .word legen 99 an B und 88 an B+4. la lädt die Adresse B, nicht den Inhalt, und kann mehrere Instruktionen brauchen. B kommt aus der Symbolansicht und muss durch vier teilbar sein.
Vor Clock bei lw a0, 4(s1): Immediate 4, Adresse B+4, Lesedaten 88, ALUSrc=1, MemtoReg=1, RegWrite=1, MemRead=1, MemWrite=0. Nach Clock ist a0=88. Die Codierung ist 0x0044A503. Danach schreibt sw a0, 0(s1) das Wort 88 an B. MemtoReg heißt im Tool nicht zwingend so.
Der Load-Pfad wird funktional beobachtet. Die 7,3 ns sind das separate Lehrtiming, keine Messung an der Leitung.
Beispiel: Datei: beispiele/mct1_vl5_load_store_trace.s.
Zum Abschluss sichern Fragen das Verständnis.

Folie 79 / 79

Q&A und Hands-on

- Ripes starten (Browser oder lokal)
- Single-Cycle Processor einstellen
- Leitungen explorieren — die CPU ist kein Mysterium mehr
- Viel Erfolg beim Hardware-Hacking



Notizen

Q&A heißt Questions and Answers. Fünf Kernaussagen: Felder wählen Register und Operation, die Datenpfade werden kombiniert, Freigaben bestimmen die Zustandsänderung, beq wählt den nächsten

PC, und Clock trennt vorbereitete Werte vom neuen Zustand.
Adresse und Load-Wert sind verschieden: B+4 ist die Adresse, 88 der Inhalt. Beim Store kommt das Datum aus RD2, obwohl ALUSrc das Immediate für die Adresse wählt. Ein sub mit Zero=1 springt nicht, wenn Branch=0 ist. Ripes liefert keine reale maximale Taktfrequenz.
Die Dateien mct1_vl5_rtype_trace.s, mct1_vl5_load_store_trace.s und mct1_vl5_branch_trace.s enthalten die Haltepunkte. Einheit 6 erzeugt die Steuersignale systematisch. Diese Vorlesung endet hier.
Beispiel: Bei gleichen Registerwerten 5 und 5 wird der beq-Zielpfad gewählt, bei 5 und 6 der Fall-through. Offen bleibt nur die konkrete Ripes-Version im Labor, nicht die Rechnung des Lehrmodells.